2024 6th International Conference on Pattern Analysis and Intelligent Systems (PAIS) | 979-8-3503-5026-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/PAIS62114.2024.10541245

Real-time Reckless Driving Detection System on a low-level computing platform: Insight from a Developing Nation

1st Md. Asifuzzaman Khan
Dept. of Electrical and Electronic Engineering
Islamic University of Technology
Dhaka, Bangladesh
asifuzzaman@iut-dhaka.edu

2nd Golam Sarowar
Dept. of Electrical and Electronic Engineering
Islamic University of Technology
Dhaka, Bangladesh
asim@iut-dhaka.edu

3rd Md. Arean Ashfaq
Dept. of Electrical and Electronic Engineering
Islamic University of Technology
Dhaka, Bangladesh
areanashfaq@iut-dhaka.edu

4th Kazi Sazzad Ahamed
Dept. of Electrical and Electronic Engineering
Islamic University of Technology
Dhaka, Bangladesh
kazisazzad@iut-dhaka.edu

***Abstract*—Reckless driving remains a significant issue in least developed countries like Bangladesh, requiring continuous monitoring of drivers' behavior to address road safety concerns effectively. State of the art solutions employ monitoring devices on-board vehicles to assess driver performance, utilizing on-board neural networks or cloud computing to analyze driving data patterns from sensor suites. However, the computational complexity and expensive processors associated with on-board neural networks make them impractical for widespread adoption in underdeveloped countries like Bangladesh. In this study, we present a proof of concept of a plug-and-play device that employs an on-board low-level processor and a cost-effective inertial measurement unit (IMU) to analyze driving behavior in real time, eliminating the need for resource heavy neural network on-board. A test bench is established to simulate various driving patterns to validate the proposed method, and the collected data is compared against published studies. The methodology is then elaborated in detail, followed by testing the entire system using the collected data and analyzing the results. The findings demonstrate that the proposed algorithmic approach in low-level computing platform is indeed effective in-terms of latency for real-time driving behavior analysis. Thus proving it promising for addressing reckless driving in developing countries.**

***Index Terms*—IMU, MEMS, GPS, ML, LSTM, CNN, DMP, DLPF, SOC.**

I. Introduction

Bangladesh a country in South Asia, is the eighth-most populous country in the world and is among the most densely populated countries. And consequently, has one of the most difficult driving conditions in the entire world. In addition, there is a lack of road planning and traffic infrastructure. Major city roads are often met with unsafe and erratic junctions. Insufficient number of lanes in major connecting roads chokes even if they barely reach rush-hour traffic. Lane allocation is non-existent or very poorly managed. This plays a key role in the non-stationary clogs seen in everyday traffic. The streets are filled with vehicles of different movement characteristics, size or propulsion method. Disruption caused by a swift lane change also results in a domino effect among vehicles that translates to major delay in junction clearance or systematic discharge of vehicles. All these factors with uneducated public bus drivers and personal chauffeurs make the roads a very unsafe place to drive or commute [1]. Though public transports are encouraged to reduce traffic jams [2]; Since its inception, Bangladesh has adopted a decentralized model for public transportation, wherein distinct privately owned entities operate along designated routes. These entities deploy multiple vehicles along the routes, accommodating passengers at key intervals based on the availability of seating within each vehicle. This raises issues like emergence of tough competition for profit, lack of liability, unsafe passenger boarding and alighting etc. Moreover, cramped seating arrangement with passengers standing causes passenger discomfort when abrupt braking or harsh turning occurs [3]. Compounded by erratic road conditions the privatization of public transportation has also engendered a regulatory vacuum, rendering traditional law enforcement evaluations unfeasible by the limited number of traffic police on duty [4]. So, an automated system that constantly monitors the public transports is a situation demanded.

II. Present method and it's limitation

Nearly all of the driving behavior recognition models utilizes machine learning [6] [7] as neural networks tend to be effective in recognizing faulty patterns [8] on the basis of a singular sensor time series data, sometimes a handful together for example- inertial measurement, GPS, Steering angle etc. [9] Either way, they recognize the discrepancy in the time series data to identify faulty patterns. However, neural networks require resource heavy computation [10] that

979-8-3503-5026-5/24/$31.00 ©2024 IEEE

hinders real time processing of motion data, which is a must for constant monitoring. Here in this paper, we propose an algorithm to replace the need of neural networks to detect faulty maneuver, measure the magnitude and log appropriate penalty in real time. The performance of this algorithm is compared to existing methods' performance in section V.
Another key factor to be considered is that the system is to be fitted on every public transport on the street, in a low-income country like Bangladesh. So, it is crucial that the unit cost to be kept bare minimum by eliminating the need of on-board powerful computing platforms like mini PCs to run neural networks.

III. Proposed system and it's features

We propose a plug and play device that can be installed easily on public transports like the local buses in Bangladesh. This device incorporates low-cost microcontrollers that assesses the vehicle motion data in real time to detect rogue driving behavior and incurs appropriate penalty. This controller is proposed to be connected to a network module to upload the penalty score to cloud. Based on this penalty, ranking of the drivers or related personnel/company may be maintained to promote good driving practices and penalize the exceeding of threshold set by traffic authority. This ultimately should reduce reckless driving and to some degree ensuring passenger comfort. At a single system cycle, the proposed system-

1) Acquires data.
2) Filters data.
3) Checks previous holding conditions.
4) Sets threshold.
5) Compares data.
6) Calculates penalty score.

The audacity evaluation is based on the amount of accumulated penalty score over a specified period of time defined by the concerned governing authority. Preferably in a month due to the small size of stored data and comparable complexity in data collection for archiving.
Flagged actions that this proposed system can identify can be divided into below three categories-

1) Abrupt braking.
2) Sharp turns
3) steep acceleration from stop.
4) collision

These flagged actions are individually identified and severity of the fault is calculated. (i.e. the maximum centrifugal acceleration in a faulty turn or linear deceleration during brake). Then penalty is calculated by multiplying a weight that is set by the governing authority with the severity value. Then all the penalty scores of different conditions are accumulated at the end of the algorithm's universal loop.

IV. Methodology

Here in this paper driving pattern is referred to the various patterns that are generated when acceleration is plotted against time for different driving maneuvers. For example: Braking, Turning, Lane changing etc. These types of patterns actually distinguish the different driving maneuvers and can quantify the extent of the maneuver. These patterns are generated based on Newton's third law of motion, that is when the vehicle accelerates to a certain direction, the IMU logs the same acceleration in opposite direction as shown in "Fig. 1".

A. Maneuver Pattern Evaluation

For detecting the flagged actions related directly to the vehicle movement, lateral and longitudinal time series acceleration data are considered as the key factors as they can tell a lot about the nature of the movement the IMU was exposed to. The time series data for hard braking vs normal braking is shown in "Fig. 2". The figure indicates that the only way a hard/emergency brake differs from a normal brake is that the longitudinal acceleration curve during hard braking has a deeper valley compared to the normal braking one. And the depth of the valley directly gives us information about the hardness of the brake. Same principle applies for the case of hard acceleration. But instead of a valley there will be a peak, and the height of the peak will represent the hardness of acceleration.

Similarly, lateral acceleration data can be used to process flagged actions related to sharp turn and abrupt lane changes as shown in the curve of "Fig. 3" for the case of swift turning vs normal turning (here the turning was for lane change).

So, using a neural network for detecting the flagged actions related to vehicle movement is not necessary as all the flagged action gives rudimentary curve shapes when acceleration is plotted against time. And they are basic enough to be processed using an algorithm only. So, there is no need to invest heavy computational power for machine learning algorithms

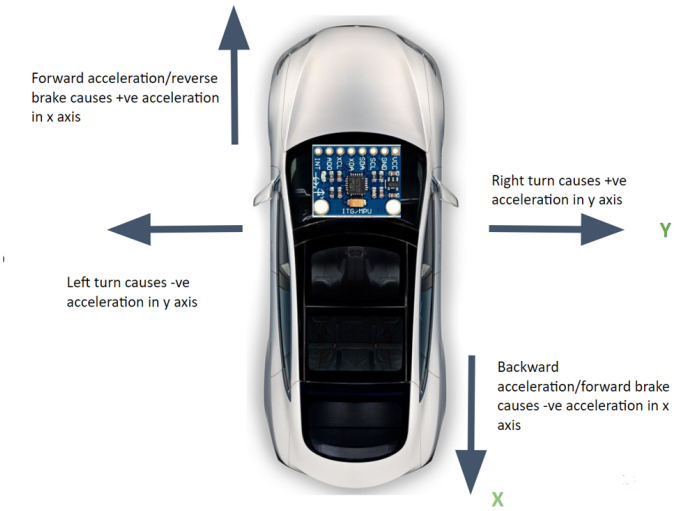

Fig. 1. Lateral (Y) and longitudinal (X) acceleration axis

B. Test bench setup

For the sake of simplicity and repetitive experiments of rough driving patterns, a test bench shown in "Fig. 4" was

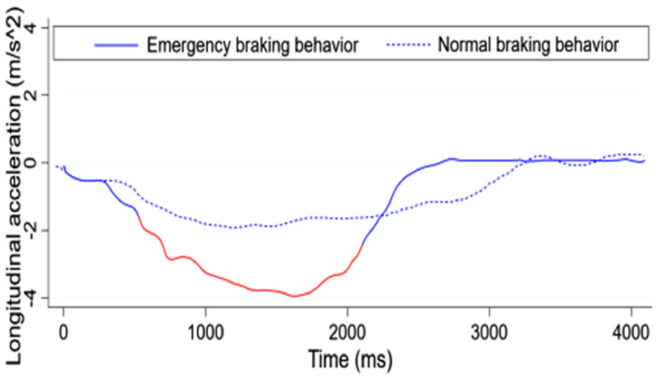

Fig. 2. Braking curve from real vehicle [6]

set up where various driving patterns could be simulated in real life in a miniature scale. To make matters reliable, a bluetooth controlled vehicle (16:1 scale) was constructed with realistic motion characteristics. Then the obtained data was compared with the published data to assess the feasibility of the test bench. The test bench features a proportional steering control for turning radius control. Semi-circular spur and pinion assembly was implemented for the steering shown in "Fig. 5". The circular spur gear was actuated using a RC servo motor with total 160° angular displacement. To simulate braking, the principal of regenerative braking is used with the only exception being the regenerated energy is dissipated by shorting the main drive motor terminals. The entire test bench is controlled over bluetooth by an android app [11]. All controls (acceleration and steering) through this app are proportional and when the accelerator slider is let go, the motor driver automatically engages brake by short circuiting the main drive motor. To simulate harsh braking the polarity of the main drive motor is altered.

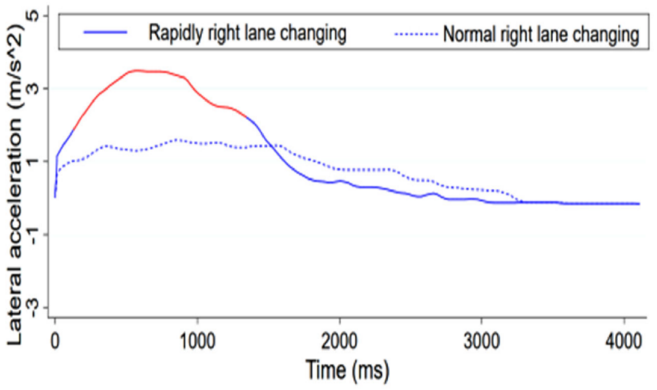

Fig. 3. Turning curve from real vehicle [6]

C. The Inertial Measurement Unit (IMU)

MPU 6050 [12] as shown in "Fig. 6" is a SOC (System On Chip) that contains a MEMS (Micro Electro Mechanical System) 3 axis (X,Y,Z) accelerometer and 3 axis (X rotation, Y rotation, Z rotation) gyroscope. This also includes a Digital Motion Processor (DMP) and a Digital Low Pass filter (DLPF). The on-board digital low pass filter (DLPF) can be configured to set necessary filtration parameters. The DLPF Bandwidth was set to 5Hz which is the minimum possible cutoff frequency for this module, considering the vibration from road surface [13]. The on-board DMP helps to read absolute acceleration on the 3 Axis. Absolute acceleration ensures that the acceleration value is compensated for gravitational acceleration on the 3 Axis, thus effectively compensating for road inclination/declination. The DMP takes care of this by calculating the gyroscopic correction angles and corresponding acceleration values.

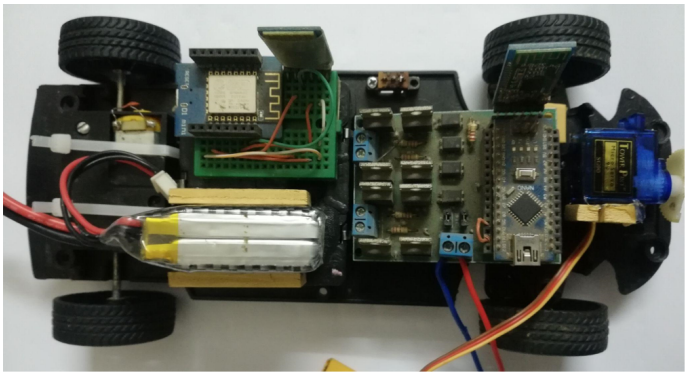

Fig. 4. The 16:1 scale test bench

D. IMU placement

A MEMS IMU works on the basis of Newton's third law, that states- every action has an equal and opposite reaction. When the module is accelerated in positive direction it logs an equal acceleration in the negative direction. So, a vehicle's lateral and longitudinal acceleration can be measured by setting the IMU axis' aligned with the corresponding axis' of the vehicle. Lateral acceleration is a crucial data using which the algorithm can detect anomalies in turning and accurately interpret the dynamics of the vehicle's motion.

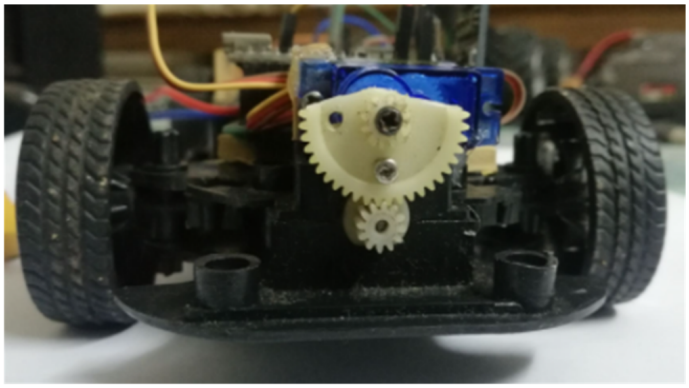

Fig. 5. Semi-circular spur and pinion assembly for steering mechanism

Fig. 6. MPU6050 Inertial Measurement Unit breakout-board

To capture maximum lateral acceleration value during turning maneuvers the IMU needs to be placed near the front axle. "Fig. 7" shows the simplified geometry behind a car turning. Here in the figure point *P* is the center of turning, line *OP* indicates turning radius, *AB* is the longitudinal axis, *RS* is the wheelbase and *OB* is the rear axle axis. To maintain the symmetry of lateral acceleration during both left and right turns, it is necessary to place the IMU along the longitudinal axis *AB*. Lateral acceleration is simply the centrifugal acceleration during turning. So, the farther the IMU is positioned from the center of turning, the greater the acceleration felt, assuming a constant vehicle speed. From the turning geometry it is clearly understood that the distance *OA* is greater than the distance *OB*. Thus, the IMU logs greater acceleration if placed at point *A* rather than at point *B*. So, it can be concluded that the IMU must be placed on the front of the vehicle along the longitudinal axis *AB* to register maximum lateral acceleration. Though this placement has minimal affect with vehicles with smaller wheelbase like, it does have a significant effect on vehicles with longer wheelbase.

E. Test bench feasibility check

To check the feasibility of our test bench the data collected from the test bench was compared with already published data from real vehicle mounted IMU sensor. The curve shown in "Fig. 2" is a published curve [6] where the time series acceleration during braking of a real vehicle is represented. The curve of "Fig. 9" was obtained from the test bench during braking.

They both have similar patterns (in terms of depth of valley) with some minor differences. For example- the published curve is filtered to be smooth and the test bench curve is not. Another difference is that the slopes of the curves do not match exactly. To be precise, the up and down slopes of the test bench appears steeper, the reason being that the test bench is much more agile compared to it's scale. However, variations in slope doesn't have any effect in our algorithm which discussed in section IV-F. Similarly, the curve during swift turning of the test bench as shown in "Fig. 8" also matches with the curve of similar maneuver in a real vehicle as shown in "Fig. 3". Based on these observation, the conclusion is reached that the test bench indeed is feasible for simulating different driving behaviors to prove the concept of such plug and play device.

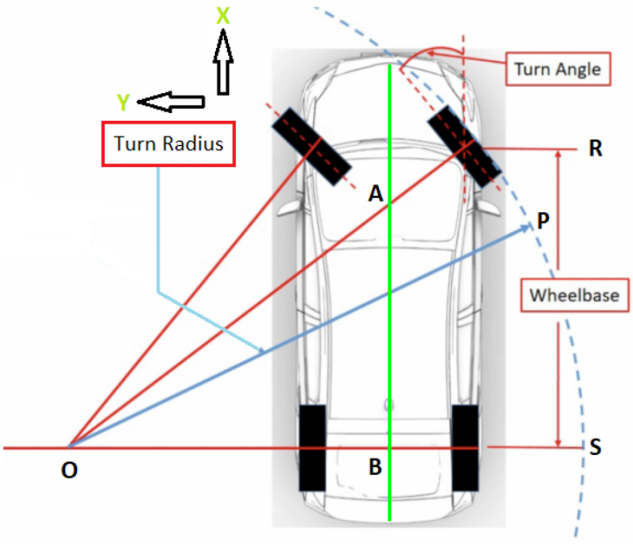

Fig. 7. Turning geometry of a vehicle

F. The motion assessment algorithm

This algorithm constantly monitors the acceleration reading from the X (longitudinal) and Y (lateral) axis of the accelerometer to detect Harsh Brake, Turn and Collision (for both axis) and to assess the magnitude. The algorithm shown in "Fig. 10" is only for longitudinal assessment(harsh acceleration/braking and longitudinal collision). However, the algorithm structure is the same for the lateral assessment (swift turning and lateral collision) portion also. Only difference being that it will be fed the lateral acceleration instead of longitudinal acceleration. Here in this algorithm, the governing variables are-

1) *AT* = Acceleration Threshold
2) *CTRT* = Cost to time Ratio Threshold. This is checked to detect collision. Higher this value, higher the probability of collision.
3) *Acceleration* = Acceleration data from the IMU (may be longitudinal or lateral)

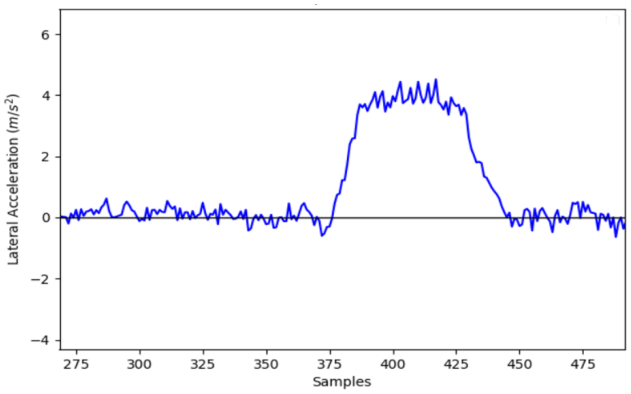

Fig. 8. Turning curve from test bench

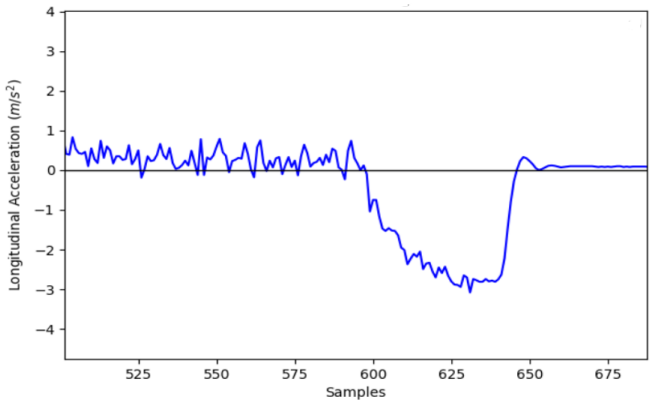

Fig. 9. Braking curve from test bench

There are 3 status variables. They are-

1) *TR* = used to check whether *AT* was crossed by *Acceleration* in the last loop. *1* means *AT* was crossed and *0* means it was not crossed.
2) *cost* = It is the integration of acceleration values since *AT* was crossed by Acceleration.
3) *time* = It represents the time elapsed since *AT* was crossed by *Acceleration*.

AT is to be set according to local laws and vehicle types/make. For example, the handbrake threshold for a mini bus and an intercity bus may not be the same. For Brake and collision assessment the threshold *AT* relates to breaking acceleration threshold and for turning it relates to centrifugal acceleration threshold due to turning. *CTRT* is also an experimental value kept during testing and subject to change according to vehicle type and route.

V. Outcome Analysis and Comparison

As the time complexity of an algorithm can give an idea about the time it takes to execute, this parameter can be considered crucial while analyzing algorithms designed for low-latency real-time application. Usually for time series data analysis (For example, time series acceleration data analysis) a special type of neural network called Recursive Neural Network (RNN) is used [17]. One of the mostly used RNN model for times series analysis is Long short term memory (LSTM). In [6], S. Jia et al.,2019 proposed a driving behavior recognition model based on the cascading of long short-term memory network and the convolutional neural network (LSTM-CNN). To find the time complexity of LSTM-CNN model first the time complexity for LSTM and CNN is to be calculated and then they can be added [14]. As LSTM is local in space and time [15], the time complexity per weight is $O(1)$. Therefore, the overall complexity of an LSTM model per time step is-

$$C_{LSTM} = O(w) \tag{1}$$

where, w is the number of weights. For CNN model as discussed in [16], the complexity is-

$$C_{CNN} = O\left(\sum_{l=1}^{d} n_{l-1}.s_l^2.n_l.m_l^2\right) \tag{2}$$

Where, d is the number of convolutional layers, n_l is the number of filters in the lth layer, n_{l-1} is the number of input channels of the lth layer, s_l is the spatial size of the filter, and m_l is the spatial size of the output feature map. Combining (1) and (2), the time complexity of LSTM-CNN model becomes-

$$C_{LSTM-CNN} = O\left(\sum_{l=1}^{d} \left(n_{l-1}.s_l^2.n_l.m_l^2\right) + w\right) \tag{3}$$

On the other hand, the algorithm presented in this paper has a time complexity of O(1). So compared to (3) it is much more suitable for real-time driving pattern detection. In order to evaluate the proposed algorithm's performance, it was run on a number of different controllers available in market as hubby-grade on their default settings. A timing benchmark was conducted by running the loop 1 million times and then calculating the average time. The results of this benchmark are presented in Table I. Column 5 shows the total run time of a single loop of the algorithm, which includes tasks such as reading sensor data from the IMU, processing longitudinal acceleration, and processing lateral acceleration. It was observed that the time taken for a single loop ranged from 2.57 to 4.6 millisecond, indicating satisfactory latency for real-time processing. The controller reads the absolute acceleration value from the IMU using the I2C protocol, as shown in column 6 of Table I. It was found that the I2C communication time accounted for at least 99.6 percent of the total loop run time (For ATmega328p), making it the primary bottleneck for the system in terms of latency. Additionally, it was observed that the full loop run time does not decrease proportionally with an increase in clock frequency or processor power, due to program compilation [17] and limiting I2c clock frequency. For example- a raspberry pi (which is an absolute overkill for this algorithm) is much more powerful than an ATmega 328p controller (1.4 GHz vs 16 MHz). But from the table it is evident that the raspberry pi's I2c read time is larger than that of Atmega 328p's. This is because, by default raspberry pi's I2c clock frequency is limited to 100 KHz. But MPU6050 supports upto 400 KHz I2c speed and ATmega 328p supports this speed. To make the read time faster one may set higher I2c clock frequency in their respective controller. However, this may result in I2c clock stretching [18] if other I2c peripherals are connected on the same bus with lower speeds. So this factor should be kept in consideration while choosing a controller.

VI. Conclusion

The proposed system has demonstrated promising results in anomaly detection in turning, linear movement and collision detection based on lateral and longitudinal acceleration measurements. In terms of time complexity, the proposed system

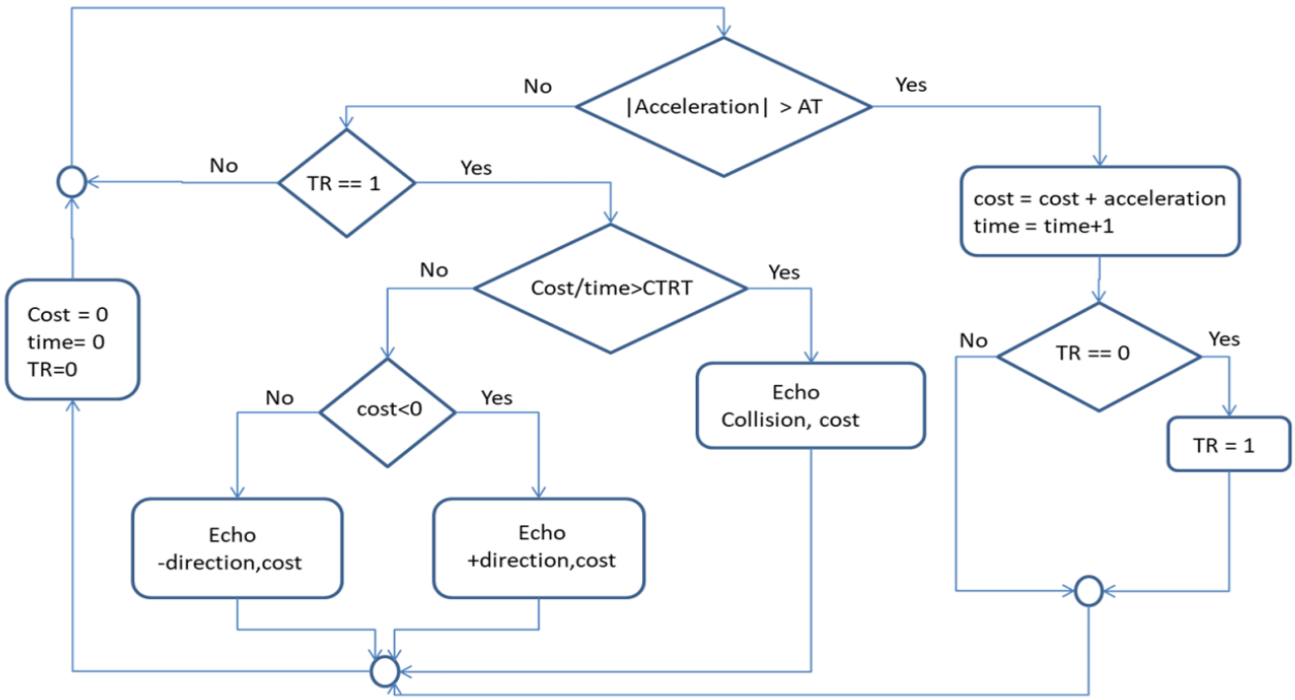

Fig. 10. The motion assessment algorithm

TABLE I
PROPOSED ALGORITHM'S PERFORMANCE ON DIFFERENT MICROCONTROLLERS

Controller	Processor	Clock speed	Processor Bit no	Full loop run time (μs)	IMU I2C read time (μs)	Cost (USD)
Arduino Uno	ATmega328p	16 MHz	8	3100	3088	∼3
Wemos D1 mini	Tensilica Diamond Standard 106Micro	80 MHz	32	3256	3252	∼3
ESP32 Devkit V1	Xtensa dual-core LX6	240 MHz	32	2570	∼2570	∼4.7
Raspberry pi 3B+	Broadcom BCM2837B0, Cortex-A53 (ARMv8)	1.4 GHz	64	4600	4596	∼43

is far more faster than not only state of the art LSTM-CNN model but also any other RNN models, due to the fact that no useful RNN model will have constant time complexity for such use cases. Making the proposed algorithm a good fit for low-latency real-time application.

The system performed well in a controlled test bench in in-door environment with smooth driving surfaces. However, real-world road scenarios present challenges due to the potential for false positives in collision detection caused by sudden road bumps. Further exploration of advanced signal processing and special mounting assembly of the IMU are required to overcome the challenges posed by road bumps. These avenues for future research will contribute to the ongoing development and refinement of the system.

REFERENCES

[1] Ahsan Ullah, AKM. "Reasons for Traffic Jam: An Observation." The Daily Star, 13 Jan 2000.

[2] A. ATM, M. Rahman, F. Islam, and S. Islam, "Encouraging Public Transport Use to Reduce Traffic Congestion in Uttara, Dhaka.," Civil Engineering Research Journal, vol. 5, no. 2, pp. 555-565, 2018, doi: 10.19080/CERJ.2018.05.555659.

[3] A. Tanim, "Oligopolistic solution in public transportation?" The Financial Express, 19 Sept 2018.

[4] J. Smita, "Corrupt, unregulated, dangerous - When will our roads get any safer?" Dhaka Tribune, 6 Feb 2022.

[5] M. Hüsken, P. Stagge,"Recurrent neural networks for time series classification," Neurocomputing, Volume 50, pp. 223–235, 2003, ISSN 0925-2312,https://doi.org/10.1016/S0925-2312(01)00706-8

[6] S. Jia, F. Hui, S. Li, X. Zhao, and A. J. Khattak, "Long short-term memory and convolutional neural network for abnormal driving behaviour recognition," IET Intelligent Transport Systems, vol. 14, no. 5, pp. 306–312, 2019. doi:10.1049/iet-its.2019.0200

[7] E. Papadimitriou, A. Argyropoulou, D. I. Tselentis and G. Yannis, "Analysis of driver behaviour through smartphone data: The case of mobile phone use while driving," Safety Science, vol. 119, pp. 91–97, 2019. doi:10.1016/j.ssci.2019.05.059

[8] M. Canizo, I. Triguero, A. Conde and E. Onieva, "Multi-head CNN–RNN for multi-time series anomaly detection: An industrial case study," Neurocomputing, vol. 363, pp. 246–260,2019, https://doi.org/10.1016/j.neucom.2019.07.034.

[9] J. Carmona, F. García, D. Martín, A. Escalera, and J. Armingol, "Data fusion for driver behaviour analysis," Sensors, vol. 15, no. 10, pp. 25968–25991, 2015. doi:10.3390/s151025968

[10] S. Michael, "On the Complexity of Computing and Learning with Multiplicative Neural Networks," Neural Computation, vol. 14, no. 2, pp. 241–301, 2002.

[11] "Joystick Control", https://sites.google.com/site/bluetoothrccar/1-parts-needed-and-the-arduino-motor-shield/6-joystick-control

[12] "MPU 6050", TDK invensense, https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6500/

[13] M. Agostinacchio, D. Ciampa, and S. Olita, "The vibrations induced

by surface irregularities in Road Pavements – A MATLAB® approach," European Transport Research Review, vol. 6, no. 3, pp. 267–275, 2013. doi:10.1007/s12544-013-0127-8

[14] E. Tsironi, P. Barros, C. Weber and S. Wermter, "An Analysis of Convolutional Long-Short Term Memory Recurrent Neural Networks for Gesture Recognition," Neurocomputing. vol. 268, 2017. doi:10.1016/j.neucom.2016.12.088.

[15] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computing, vol. 9, pp. 1735–1780, 1997. doi:10.1162/neco.1997.9.8.1735.

[16] K. He and J. Sun, "Convolutional neural networks at constrained time cost," in: Proceeding of the 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5353–5360, 2015. doi:10.1109/CVPR.2015.7299173.

[17] F. Zehra, M. Javed, D. Khan and M. Pasha, "Comparative Analysis of C++ and Python in Terms of Memory and Time," Preprints, 2020, 2020120516. doi:/10.20944/preprints202012.0516.v1

[18] "I2C clock stretching" , Texas Instruments, 22 DEC 2021, https://www.ti.com/video/6288224255001